

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO

ORIENTADO POR OBJETOS

Diseño y Pruebas.

2026-2 — Laboratorio 2/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Desarrollar una aplicación aplicando Desarrollo Orientado al Comportamiento (BDD) y Desarrollo Dirigido por Modelos (MDD).
2. Realizar diseños (ingeniería directa e inversa) utilizando una herramienta de modelado (astah)
3. Manejar pruebas de unidad usando un framework ([junit](#))
4. Apropiar nuevas clases consultando sus especificaciones ([API java](#))
5. Experimentar las prácticas XP:
Designing. Use [CRC](#) cards for design sessions.
Testing. All code must have [unit tests](#).

ENTREGA

- Incluyan en un archivo [.zip](#) los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ) Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO Objetivo: Desarrollar una versión simplificada de [Tunes](#): [miniTunes](#)

[iTunes](#) es una aplicación para administrar colecciones musicales basada en estructuras como canciones, álbumes y listas de reproducción, que permiten organizar y manipular información musical de forma eficiente.

En este laboratorio desarrollaremos una versión simplificada de [iTunes](#) utilizando únicamente listas de reproducción

Una lista de reproducción es una colección ordenada de canciones sobre la cual pueden realizarse operaciones de consulta, selección y combinación.

En [miniTunes](#) cada canción estará caracterizada por su título, artista, género, duración y calificación. [iTunes/Music Tutorial](#)



PARTE I. Conociendo el proyecto

A Revisando línea base [En [lab02.pdf](#)]

1. El proyecto “[miniTunes](#)” contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de
(a) directorios

La estructura de directorios es:

C:.

```
└─miniTunes
    └─miniTunes
        └─miniTunes
            └─miniTunes
            └─miniTunes
            └─miniTunes
            └─package
```

- └─Playlist
- └─Playlist
- └─Playlist
- └─PlaylistTest
- └─PlaylistTest
- └─PlaylistTest
- └─README
- └─doc
 - └─allclasses-frame
 - └─allclasses-noframe
 - └─constant-values
 - └─help-doc
 - └─index
 - └─index-all
 - └─logfile
 - └─overview-tree
 - └─package-frame
 - └─package-list
 - └─package-summary
 - └─package-tree
 - └─script
 - └─stylesheet
 - └─resources
 - └─inherit

(b) de las extensiones de los archivos.

C:.

- └─miniTunes.
 - └─miniTunes.asta
 - └─miniTunes
 - └─miniTunes.class
 - └─miniTunes.java
 - └─miniTunes.ctxt
 - └─package.bluej
 - └─Playlist.class
 - └─Playlist.java
 - └─Playlist.ctxt
 - └─PlaylistTest.class
 - └─PlaylistTest.java
 - └─PlaylistTest.ctxt
 - └─README.TXT
 - └─doc
 - └─allclasses-frame.html
 - └─allclasses-noframe.html
 - └─constant-values.html

- └─ help-doc.html
- └─ index.html
- └─ index-all.html
- └─ logfile.txt
- └─ overview-tree.html
- └─ package-frame.html
- └─ package-list
- └─ package-summary.html
- └─ package-tree.html
- └─ script.js
- └─ stylesheet.css
- └─ resources
 - └─ inherit.gif

2. Exploren el proyecto en [BlueJ](#)

(a) ¿Cuántas clases tiene?

Tiene 4 clases contando String.

(b) ¿Cuál es la relación entre ellas?

Tienen dos relaciones de uso:

-MiniTunes a Playlist.

-PlaylistTest a Playlist.

(c) ¿Cuál es la clase principal de la aplicación?

La clase principal de la aplicación es MiniTunes.

(d) ¿Cómo la reconocen?

La clase principal es a la que le colaboran las demás clases.

(e) ¿Cuál es la clase “diferente”?

La clase diferente es PlaylistTest.

(f) ¿Cuál es su propósito?

El propósito de dicha clase es establecer pruebas rápidas de funcionamiento, conocidas como pruebas unitarias.

Para las siguientes dos preguntas sólo consideren las clases “normales”:

3. Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)

Lo que le falta según la documentación:

- Ninguna de los dos tiene atributos.

- MiniTunes

- Documentación a los métodos.
- Detalles del constructor.
- Decir que retorna cada método.
- Playlist
 - Le falta documentación a la clase.
 - Documentación a los métodos.
- Le falta decir qué retorna los métodos.

4. Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones:

(a) la primera, atributos y métodos, y la segunda,

	Playlist	MiniTunes
<i>Atributos</i>	No tiene atributos.	Tiene solo un atributo.
<i>Métodos</i>	No tiene código implementado.	No tiene código implementado.

(b) código, documentación y comentarios)

	Playlist	MiniTunes
<i>Documentación/comentarios</i>	No tiene documentada el encabezado de la clase.	No tiene especificación del encabezado de la clase.
<i>Código</i>	- No tiene código implementado. - Tiene en un solo método la especificación de lo que hace.	- No tiene código implementado. - Tiene comentarios más no documentación.

(c) ¿Qué diferencia hay entre el código, la documentación y los comentarios?

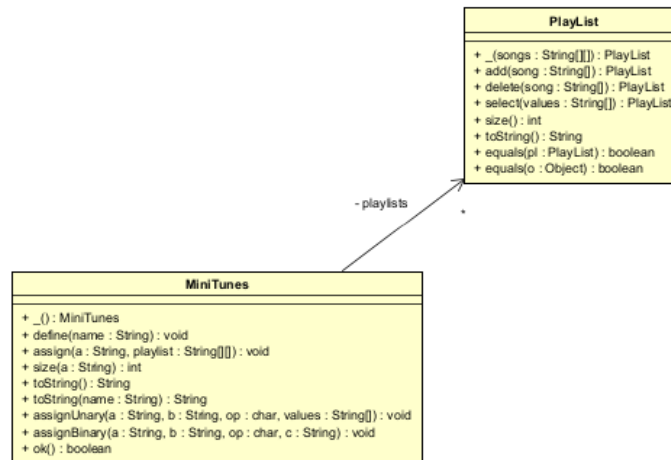
La diferencia entre la documentación y los comentarios es la sintaxis y para que se utiliza cada uno. Los comentarios están orientados a otros programadores mientras analizan la estructura lógica de la clase. La documentación genera un archivo HTML que sirve para entender la estructura general y el porqué de cada clase y que métodos contiene, con información de dichos métodos. Por ejemplo, en MiniTunes tiene documentada la clase para saber quién es el autor, pero comentado el que hace los métodos y en HTML solo se ve la documentación de la clase.

B Realizando ingeniería inversa

[En [lab02.pdf](#) [miniTunes.astah](#)]

MDD: MODEL DRIVEN DEVELOPMENT

1. Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas)



(a) ¿Cuáles contenedores están definidos?

Los contenedores definidos en el diagrama anterior son:

- Playlist de la multiplicidad entre la clase MiniTunes y Playlist.
- Vectores como `String []` y vectores de vectores como `String [][]`, en MiniTunes, los métodos que contienen como parámetros vectores son:
 1. Assign, el parámetro playlist (TreeMap).
- En Playlist los vectores son:
 1. Playlist, el parámetro songs.
 2. Add, el parámetro song.
 3. Delete, el parámetro song.
 4. Select, el parámetro values.

(b) ¿Qué diferencias hay entre el nuevo contenedor y el ArrayList y el vector `[]` que conocemos? Consulte el API de java.

Las dos clases utilizan a List, ósea son un arreglo dinámico internamente, ósea a nivel de estructura de datos son exactamente el mismo. Los dos utilizan un arreglo nativo para almacenar bloques continuos en memoria y un contador entero que nos indica cuantos elementos tenemos en ese arreglo.

La diferencia entre un ArrayList y el vector está en los algoritmos internos que utilizan para manipular su estructura de datos.

Un array es una estructura de datos de tamaño fijo que se utiliza para almacenar múltiples elementos del mismo tipo de datos en ubicaciones de memoria contiguas. Se accede a cada elemento mediante su índice, comenzando desde 0.

- Los elementos se almacenan en ubicaciones de memoria secuenciales, lo que permite un acceso rápido mediante índices.
- Admite tanto tipos de datos primitivos (int, char, double, etc.) como objetos.
- Se puede acceder a la longitud de un array mediante la propiedad `length`.

ArrayList es una implementación de la interfaz List que permite el almacenamiento dinámico de elementos, ya que proporciona acceso aleatorio rápido y una gestión flexible del tamaño.

- Mantiene el orden de inserción de los elementos.
- No está sincronizado, por lo que no es seguro para subprocesos.
- Crece dinámicamente a medida que se añaden elementos.

Vector es una implementación de arreglo dinámico en Java que crece automáticamente al agregar elementos. Es similar a ArrayList, pero es sincronizado y seguro para subprocesos.

- Vector es una clase heredada introducida en Java 1.0 antes del Collection Framework.
- Mantiene el orden de inserción de los elementos.
- Más lento que ArrayList debido a la sobrecarga de sincronización.

2. En el nuevo contenedor,

(a) ¿Cómo adicionamos un elemento?

El método que nos permite adicionar un elemento es: `put(K key, V value)`

(b) ¿Cómo lo consultamos?

El método para consultar todo el árbol: `entrySet()`

(c) ¿Cómo lo eliminamos?

Para eliminar todos los elementos se utiliza el método: `clear()`

Para eliminar un nodo en específico, utilizamos el método: `remove(Object key)`

PARTE II. Aprendiendo a probar

A Conociendo Pruebas en BlueJ [En lab02.pdf *.java]

BDD

Para poder cumplir con las prácticas XP vamos a aprender a realizar las pruebas de unidad usando las herramientas apropiadas. Para eso implementaremos algunos métodos en la clase [Playlist](#).

1. Revisen el código de la clase [PlaylistTest](#)

(a) ¿cuáles etiquetas tiene (componentes con símbolo @)?

Las etiquetas son las siguientes

- @before
- @test
- @after

@before: indica que el método marcado debe ejecutarse antes de cada una de las pruebas que están ejecutados como @test.

@test: marca el método que actúa como prueba unitaria.

@after: Se ejecuta después de las pruebas unitarias.

(b) ¿cuántos métodos tiene?

Tiene 6 métodos.

- setUp()
- shouldCreateAEmptyPlaylist()
- shouldCreateAPlaylist()
- shouldNotCreateABadPlaylist()
- shouldRecognizeEqualPlaylists()
- tearDown()

(c) ¿cuántos métodos son de prueba?

Son 4 los métodos de prueba.

- shouldCreateAEmptyPlaylist()
- shouldCreateAPlaylist()
- shouldNotCreateABadPlaylist()
- shouldRecognizeEqualPlaylists()

(d) ¿cómo los reconocen?

Porque en la parte superior del encabezado del método tienen la etiqueta @test.

2. Ejecuten los tests de la clase [PlaylistTest](#). (click derecho sobre la clase, Test All)

(a) ¿cuántas pruebas se ejecutan?

Se ejecutan 4 pruebas.

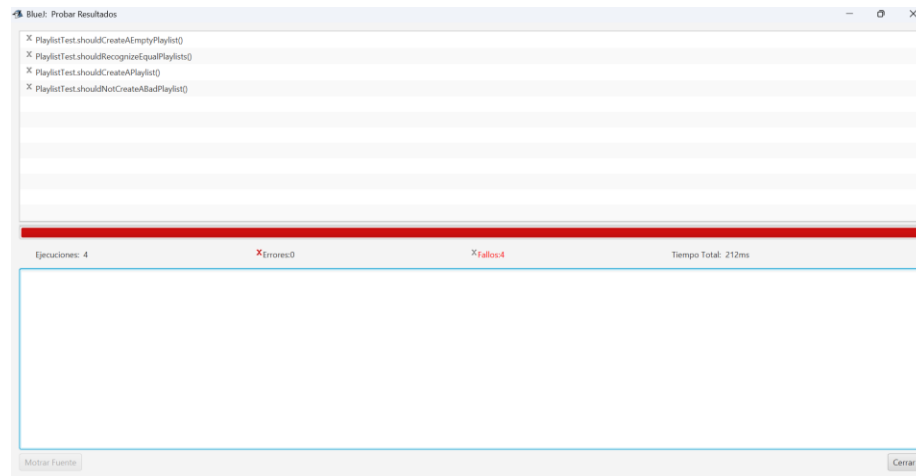
(b) ¿cuántas pasan?

Ninguna prueba pasa.

(c) ¿por qué?

Porque los métodos que prueban de la clase Playlist no están implementados por lo tanto no van acorde a los resultados de la prueba, es decir no van acorde al resultado de los asserts.

(d) Capturen la pantalla.



3. Estudian las etiquetas encontradas en 1. Expliquen en sus palabras su significado.
Al observar el método se ve lo siguiente:

```
/**
 * Sets up the test fixture.
 * Called before every test case method.
 */
@Before
public void setUp(){
}
```

Como se investigó y dice en la documentación de este método se ejecuta o es llamado este método antes de ejecutar cada prueba, esto hace la etiqueta @before.

```

6
7 public class PlaylistTest{
8
9
10
11 /**
12  * Sets up the test fixture.
13  * Called before every test case method.
14  */
15 @Before
16 public void setUp(){
17
18 }
19
20
21 @Test
22 public void shouldCreateAEmptyPlaylist(){
23     String [][] songs = {};
24     Playlist pl=new Playlist(songs);
25     assertEquals(0, pl.size());
26 }
27
28 @Test
29 public void shouldCreateAPlaylist(){
30     String [][] songs=
31         {{"One", "U2", "Rock", "4", "*****"},
32          {"Numb", "Linkin Park", "Rock", "3", null},
33          {"Alive", "Pearl Jam", "Rock", "5", "*****"},
34          {"Creep", "Radiohead", "Rock", null, "*****"},
35          {"Dreams", "Fleetwood Mac", null, "4", "*****"}};
36     Playlist pl=new Playlist(songs);
37     assertEquals(5, pl.size());
38 }
39
40 @Test
41 public void shouldNotCreateABadPlaylist(){
42     String [][] songs=
43         {{"One", "U2", "Rock", "4", "*****"}

```

@test: marca el método que actúa como prueba unitaria.

```

68
69 /**
70  * Tears down the test fixture.
71  *
72  * Called after every test case method.
73  */
74 @After
75 public void tearDown(){
76
77 }
78

```

@after: Se ejecuta después de las pruebas unitarias.

4. Estudie los métodos [assertTrue](#), [assertFalse](#), [assertEquals](#), [assertNull](#) y [fail](#) de la clase [Assert](#) del API JUnit

1. Explique en sus palabras que hace cada uno de ellos.

Lo que hace [assertTrue](#) es verificar si el parametro que se le pasa es verdadero o no, por lo que siempre se le pasa un dato tipo booleano, es un método monádico, o unaria, de un solo parámetro, el segundo parámetro lo recibe para indicar el mensaje de error que da si llega a fallar esta verificación.

El método [assertFalse](#) de manera similar que el anterior método es un método de un solo parámetro, el cual verifica si lo que se le pasa como argumento es falso, su segundo parámetro se puede llenar, pero no afecta la prueba o verificación, es el mensaje que se le pone cuando falla.

Ahora [assertEquals](#) compara dos objetos y verifica si son iguales por medio de los métodos de los mismos objetos llamados equals(), si da verdadero es porque son iguales y si da falso es que discrepan, sus parámetros lo hacen diádico, o binario, el cual como primer dato recibe el objeto esperado y como segundo dato recibe el objeto real o final.

Por otro lado, se tiene `assertNull` esta verificación salta aprobada si el objeto que se le pasa como parámetro es nulo (`null`), más si no es nulo el objeto no es aprobada esta prueba.

Por último y no menos importante `fail`, esta palabra clave lo que hace es hacer fallar una prueba, se usa normalmente para evaluar condiciones no deseadas del código, falla la prueba y automáticamente detiene la ejecución de aquella prueba.

5. Investiguen y expliquen la diferencia que entre un fallo y un error en `Junit`. Escriba código, usando los métodos del punto 4., para codificar los siguientes tres casos de prueba y lograr que se comporten como lo prometen `shouldPass`, `shouldFail`, `shouldErr`.

Los fallos suceden cuando las verificaciones (asserts) de las pruebas no pasan mientras que un error es cuando algo externo a las aserciones falla.

Ahora los métodos de prueba son los siguientes:

```
@Test
public void shouldPass() {
    String proof1 = "youpass", proof2 = "youpass";
    assertEquals(proof1, proof2);
}

@Test
public void shouldFail() {
    fail();
}

@Test
public void shouldError() {
    String variable = null;
    variable.charAt(5);
}
```

Al ejecutarlos aparece lo siguiente

```
✗ PlaylistTest.shouldError()
✗ PlaylistTest.shouldFail()
✔ PlaylistTest.shouldPass()
```

B Practicando Pruebas en BlueJ [En lab02.pdf *.java]

BDD

Ahora vamos a escribir el código necesario para que las pruebas de pasen `PlaylistTest`.

1. (a) Determinen los atributos de la clase `Playlist`.

Los atributos seleccionados son los siguientes:

```
public class Playlist {  
    private String[][] songs;  
    private int size;
```

- (b) Justifiquen la selección.

Por los argumentos de los métodos o los mismos métodos es que se escogen estos atributos, por ejemplo el atributo `songs` se escoge porque en el constructor hay un parametro que recibe aquello, como se muestra a continuación:

```
public Playlist(String [][] songs){  
}
```

El atributo `size` se escoge por el retorno y nombre d este método

```
public int size(){  
    return size;  
}
```

2. (a) Determinen el invariante de la clase `Playlist`.

El invariante que siempre se debe cumplir es que el atributo `size` siempre es mayor que 0.

También se definen como las siguientes reglas para verificar la integridad de la tabla de los datos

- ☐ El título y el artista son obligatorios. El género, la duración y la calificación pueden ser desconocidos.
- ☐ La combinación (título, artista) debe ser única. Dos canciones no pueden tener el mismo título y artista.
- ☐ La duración (minutos) debe estar entre 1 y 9.
- ☐ La calificación debe estar entre * y *****.

- (b) Justifiquen la decisión.

Esto es porque es necesario operar con números positivos para poder acceder al último índice de las filas del atributo `song`.

Con respecto a las reglas de la tabla esto se permite para poder verificar que se puedan tener datos estandarizados.

3. Implementen los métodos de `Playlist` necesarios para pasar todas las pruebas definidas.

- (a) ¿Cuáles métodos implementaron?

Se perfeccionó el método constructor de `Playlist`, se implementó `toString` y `equals`:

```

public Playlist(String [][] songs){
    ArrayList<String[]> validSongs = getValidSongs(songs);
    int sizeValidSongs = validSongs.size();
    this.songs = new String[sizeValidSongs][5];
    for (int i = 0; i < sizeValidSongs; i++) {
        this.songs[i] = validSongs.get(i);
    }
    size = this.songs.length;
}

private ArrayList<String[]> getValidSongs(String [][] songsToVerify) {
    ArrayList<String[]> validSong = new ArrayList<>();
    boolean isThereSomeNull; String currentSong;
    for (int i = 0; i < songsToVerify.length; i++) {
        isThereSomeNull = false;
        for (int j = 0; j < songsToVerify[i].length; j++) {
            currentSong = songsToVerify[i][j];
            if (j == 0) {
                if (currentSong == null) {
                    isThereSomeNull = true;
                    continue;
                }
            } else if (j == 1) {
                if (currentSong == null) {
                    isThereSomeNull = true;
                    continue;
                }
            }
        }
        if (!isThereSomeNull) {
            validSong.add(songsToVerify[i]);
        }
    }
    return validSong;
}

```

```

/**
 * Verify if is the same playlist
 */
public boolean equals(Playlist pl){
    String[][] songsToCompare = pl.getSongs();
    if (this.size != pl.size()) return false;
    String nameSong, nameSongToCompare;
    boolean isSameSong;
    for (int i = 0; i < songs.length; i++) {
        for (int j = 0; j < songs[i].length; j++) {
            isSameSong = true;
            if ((songs[i][j]==null && songsToCompare[i][j]!=null) ||
                (songs[i][j]!=null && songsToCompare[i][j]==null)) return false; // Si alguna de las dos es nula
            if (songs[i][j]!=null && songsToCompare[i][j]!=null) {
                nameSong = songs[i][j].trim().replaceAll("\\s+", " ");
                if (j == 4) {
                    nameSongToCompare = songsToCompare[i][j].trim().replaceAll("\\s+", "");
                } else {
                    nameSongToCompare = songsToCompare[i][j].trim().replaceAll("\\s+", " ");
                }
                isSameSong = nameSong.equalsIgnoreCase(nameSongToCompare);
            }
            if (!isSameSong) {
                return false;
            }
        }
    }
    return true;
}

```

```

/** Songs are in uppercase with unnecessary spaces removed.
 *Columns are aligned and separated by three spaces.
 *TITLE  ARTIST      GENRE  DURATION  RATING
 *ONE    U2          ROCK   4        *****
 *NUMB   LINKIN PARK  ROCK   3        ****
 *ALIVE   PEARL JAM   ROCK   5        ****
 *CREEP   RADIOHEAD   ROCK   .        *****
 *DREAMS FLEETWOOD MAC .        4        ****
 *@return
 */
public String toString() {
    String messageToShow = String.format("%-3s  %-3s  %-3s  %-3s  %-3s",
        "TITLE", "ARTIST", "GENRE", "DURATION", "RATING")+"\n";

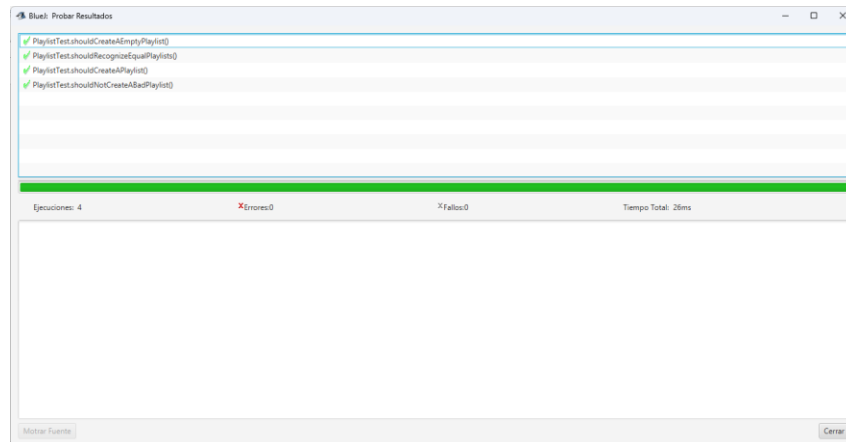
    String title, artist, genre, duration, rating;
    boolean isDurationANumber, isRatingANumber;
    for (int i = 0; i < songs.length; i++) {
        isDurationANumber = true; isRatingANumber = false;
        title = songs[i][0]; artist = songs[i][1]; genre = songs[i][2];
        title = prepareString(title);
        artist = prepareString(artist);
        genre = prepareString(genre);

        duration = songs[i][3]; rating = songs[i][4];
        if (duration != null) isDurationANumber = duration.matches("\\d+");
        if (rating != null) {
            isRatingANumber = rating.matches("\\d+");
            rating = rating.trim().replaceAll("\\s+", "");
        }

        if (!isDurationANumber) duration = null;
        if (!isRatingANumber) rating = convertNumberToRating(rating);
        messageToShow+=String.format("%-3s  %-3s  %-3s  %-3s  %-3s",
            title, artist, genre, duration, rating)+"\n";
    }
    return messageToShow;
}

```

4. Capturen los resultados de las pruebas de unidad.



PARTE III. Desarrollando el proyecto **miniTunes**. [En [lab02.pdf](#) [miniTunes.astah](#) *.java]

MDD-BDD

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

En cada ciclo deben realizar los pasos definidos a continuación.

- I Definir los métodos base de correspondientes al mini-ciclo actual.

Playlist
Ciclo 1
• Select() • Add() • Delete()

- II Definir y programar los casos de prueba de esos métodos.

Piensen en los debería y los noDebería ([should...](#) y [shouldNot...](#))

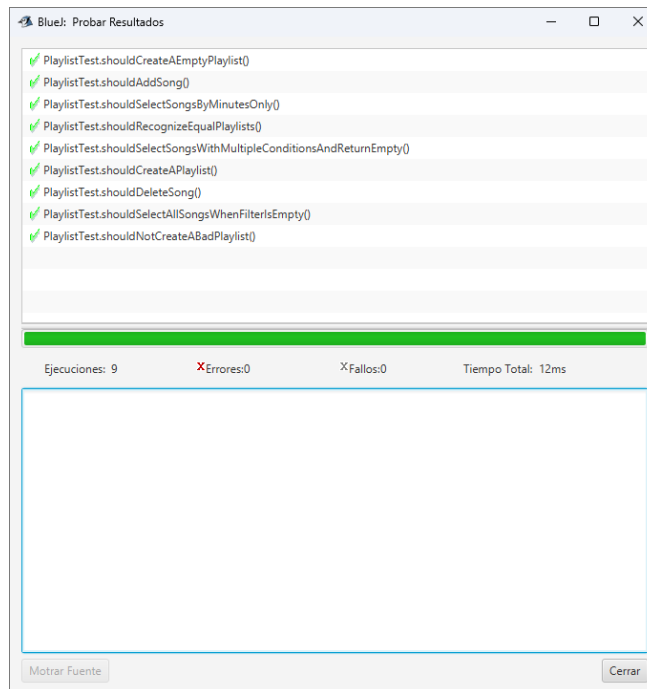
En Playlist:

- ShouldAddSong
- ShouldDeleteSong
- ShouldSelectSongsWithMultipleConditionsAndReturnEmpty
- ShouldSelectSongsByMinutesOnly
- shouldSelectAllSongsWhenFilterIsEmpty

- III Diseñar los métodos

Usen diagramas de secuencia. En astah, creen el diagrama sobre el método correspondiente.

- IV Escribir el código correspondiente (no olvide la documentación)
- V Ejecutar las pruebas de unidad (vuelva a 3 (a veces a 2), si no están en verde)
- Playlist:



vi Completar la tabla de clases y métodos. (Al final del documento)

Ciclo 1 : Operaciones básicas: definir el nombre de una lista, asignar a un nombre una lista, consultar nombres de listas y consultar las canciones de una lista.

Ciclo 2: Operaciones unarias: adicionar una canción, eliminar una canción y seleccionar canciones dada una condición.

Ciclo 3 : Operaciones binarias: unión, intersección y diferencia.

BONO Ciclo 4 : Defina e implemente tres nuevas operaciones

Completan la siguiente tabla indicando el número de ciclo y los métodos asociados de cada clase.

Ciclo	MiniTunes	MiniTunesTest
1	• <code>_()</code> • <code>Size()</code> • <code>Assign()</code> • <code>Define()</code> • <code>ToString()</code> • <code>ToString(name)</code>	
2	• <code>AssignUnary()</code>	
3	• <code>AssignUnary()</code> • <code>AssignBinary()</code>	

	• Ok()	
--	--	--

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

15 Villagran

14 Gualdron

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Terminado, se hizo lo que se exige en este laboratorio.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

La más importante fue Testing, porque esta práctica da un sustento de cómo debe de actuar o comportarse el programa lo cual da solidez en probar que el software se comporta de una manera en específica.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Hacer pruebas de unidad de manera correcta, esto permite que la prueba verifique los requerimientos que exigen en este laboratorio.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Comprender código del compañero porque puede que sea muy elaborado el código, con pocas explicaciones, lo único que se hizo fue volver a leer código.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

La comunicación en la organización de las partes de este laboratorio, para mejorar y perfeccionar esta nos comprometemos a explicarnos partes del código que no entendamos.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- JUnit. (2016). *Annotation Type Before (JUnit API version 4.12)*. <https://junit.org/junit4/javadoc/4.12/org/junit/Before.html>
- JUnit. (s. f.). *Class Assert (JUnit API)*. <https://junit.org/junit4/javadoc/latest/org/junit/Assert.html>
- froadie. (2010, 6 de agosto). *What's the difference between failure and error in JUnit?* Stack Overflow. <https://stackoverflow.com/questions/3425995/whats-the-difference-between-failure-and-error-in-junit>
- JUnit5/Jupiter(Claseorg.junit.jupiter.api.Assertions): <https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>
- Oracle.(s.f.).ClassTreeMap(JavaSE21&JDK21). <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html>
- GeeksforGeeks. (s.f.). *Array vs ArrayList in Java*. <https://www.geeksforgeeks.org/java/array-vs-arraylist-in-java/>
- GeeksforGeeks. (s.f.). *Vector vs ArrayList in Java*. <https://www.geeksforgeeks.org/java/vector-vs-arraylist-java/>
- Oracle. (s.f.). *Package java.util (Java SE 21 & JDK 21)*. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/package-summary.html>